

Drzewa wszystkich najkrótszych ścieżek

Algorytm Dijkstry – implementacja UPC++

Jakub Szydelko
Jakub Urbański

2025/2026

Spis treści

1	Wprowadzenie	3
1.1	Cel projektu	3
1.2	Sformułowanie problemu	3
2	Algorytm Dijkstry	4
2.1	Wersja sekwencyjna	4
2.2	Złożoność wersji sekwencyjnej	5
3	Zrównoleglenie z wykorzystaniem UPC++	6
3.1	Model PGAS	6
3.2	Strategia zrównoleglenia	6
3.3	Podział danych	6
3.4	Dystrybucja danych wejściowych	6
3.5	Globalna selekcja minimum	7
3.6	Relaksacja krawędzi	7
3.7	Agregacja wyników	7
3.8	Synchronizacja	8
3.9	Złożoność i skalowanie	8
4	Architektura projektu	9
4.1	Struktura projektu	9
4.2	Struktura kodu	9
4.3	Format danych wejściowych	10
4.4	Format danych wyjściowych	10
4.5	Struktura pliku JSON	10
4.6	Pomiar czasu wykonania	11
5	Schemat blokowy programu	11
5.1	Schemat głównej pętli programu	11
5.2	Schemat funkcji <code>DijkstraUPCXX()</code>	13

6	Opis implementacji	15
6.1	Struktura kodu źródłowego	15
6.2	Struktury pomocnicze	15
6.3	Wczytywanie danych i zapis wyników	15
6.4	Implementacja algorytmu Dijkstry	16
6.5	Agregacja wyników	16
6.6	Funkcja główna programu	16
7	Budowa i obsługa programu	17
7.1	Wymagania i kompilacja	17
7.2	Uruchamianie programu	17
7.3	Dodatkowe reguły Makefile	18
7.4	Wyniki działania	18
7.5	Obsługa błędów	18
7.6	Przebieg działania programu	19
8	Przykłady działania i analiza wyników	20
8.1	Przykładowe dane i uruchomienie	20
8.2	Przykładowe wyniki	20
8.3	Analiza komunikacji i skalowalności	21
8.4	Porównanie z implementacją MPI	21
9	Podsumowanie	21

1 Wprowadzenie

1.1 Cel projektu

Celem projektu jest równoległa implementacja algorytmu Dijkstry wyznaczającego drzewa najkrótszych ścieżek w grafie ważonym. Dla każdego wierzchołka źródłowego wyznaczane są długości najkrótszych ścieżek do pozostałych wierzchołków oraz odpowiadające im drzewa najkrótszych ścieżek reprezentowane za pomocą tablicy poprzedników.

Implementacja została wykonana z wykorzystaniem biblioteki UPC++ (Unified Parallel C++), realizującej model programowania PGAS (Partitioned Global Address Space). Biblioteka ta udostępnia mechanizmy globalnej przestrzeni adresowej oraz operacje komunikacyjne, które umożliwiają efektywne wykonywanie obliczeń równoległych.

W ramach projektu opracowano równoległą wersję algorytmu wykorzystującą operacje `broadcast()`, `reduce_all()` oraz `rput()`, a także przeanalizowano możliwości skalowania rozwiązania dla różnych liczb procesów. Wyniki programu zapisywane są na standardowym wyjściu oraz w formacie JSON.

1.2 Sformułowanie problemu

Niech

$$G = (V, E, w)$$

oznacza graf ważony, gdzie V jest zbiorem wierzchołków, E zbiorem krawędzi, a w funkcją przypisującą nieujemne wagi krawędziom.

Graf przekazywany jest do programu w postaci macierzy sąsiedztwa

$$W \in \mathbb{Z}^{n \times n},$$

w której dodatnia wartość oznacza wagę krawędzi, natomiast wartość -1 oznacza brak bezpośredniego połączenia pomiędzy wierzchołkami.

Dla każdego źródła

$$s \in \{0, 1, \dots, n-1\}$$

należy wyznaczyć długości najkrótszych ścieżek do wszystkich wierzchołków oraz odpowiadające im tablice poprzedników.

Problem ten można traktować jako wykonanie algorytmu Dijkstry niezależnie dla każdego wierzchołka źródłowego, co dla grafu o n wierzchołkach oznacza konieczność przeprowadzenia n przebiegów algorytmu.

Ze względu na wysoką złożoność obliczeniową naturalnym kierunkiem optymalizacji jest wykorzystanie obliczeń równoległych. W prezentowanej implementacji zastosowano podział zbioru wierzchołków pomiędzy procesy UPC++, które wspólnie uczestniczą w wyznaczaniu globalnego minimum oraz relaksacji krawędzi. Pozwala to rozłożyć obciążenie obliczeniowe pomiędzy wiele procesów przy zachowaniu poprawności algorytmu.

2 Algorytm Dijkstry

2.1 Wersja sekwencyjna

Algorytm Dijkstry wyznacza najkrótsze ścieżki w grafie ważonym o nieujemnych wagach krawędzi. Dla zadanego wierzchołka źródłowego s oblicza długości najkrótszych ścieżek do wszystkich pozostałych wierzchołków.

W implementacji wykorzystywane są trzy tablice:

- $l[v]$ – aktualnie znana odległość od źródła,
- $parent[v]$ – poprzednik na najkrótszej ścieżce,
- $visited[v]$ – informacja o odwiedzeniu wierzchołka.

Na początku wszystkie odległości ustawiane są na wartość nieskończoną, natomiast dla źródła:

$$l[s] = 0.$$

W każdej iteracji wybierany jest nieodwiedzony wierzchołek o najmniejszej odległości, a następnie wykonywana jest relaksacja jego krawędzi. Jeżeli dla krawędzi (u, v) zachodzi:

$$l[u] + w(u, v) < l[v],$$

to aktualizowane są wartości:

$$l[v] = l[u] + w(u, v),$$

$$parent[v] = u.$$

Proces powtarzany jest aż do wyznaczenia najkrótszych ścieżek do wszystkich osiągalnych wierzchołków.

Pseudokod algorytmu przedstawiono poniżej.

Listing 1: Pseudokod algorytmu Dijkstry

```

DIJKSTRA( $G, s$ )

for each vertex  $v$ :
     $l[v] \leftarrow \text{INF}$ 
     $\text{parent}[v] \leftarrow -1$ 
     $\text{visited}[v] \leftarrow \text{false}$ 

 $l[s] \leftarrow 0$ 

while exists unvisited vertex:

     $u \leftarrow \text{argmin}(l[v])$ 

     $\text{visited}[u] \leftarrow \text{true}$ 

    for each neighbour  $v$  of  $u$ :

        if  $l[u] + w(u, v) < l[v]$ :

             $l[v] \leftarrow l[u] + w(u, v)$ 
             $\text{parent}[v] \leftarrow u$ 

```

Wynikiem działania algorytmu są tablice odległości oraz poprzedników, na podstawie których można odtworzyć pełne ścieżki pomiędzy wierzchołkami.

2.2 Złożoność wersji sekwencyjnej

W projekcie zastosowano implementację wykorzystującą macierz sąsiedztwa oraz liniowe wyszukiwanie minimum.

Dla pojedynczego przebiegu wykonywanych jest n iteracji, a każda z nich wymaga wyszukania minimum i relaksacji krawędzi w czasie $O(n)$. Ostatecznie prowadzi to do złożoności:

$$T(n) = O(n^2).$$

Ponieważ algorytm wykonywany jest niezależnie dla wszystkich wierzchołków źródłowych, całkowita złożoność problemu wynosi:

$$O(n) \cdot O(n^2) = O(n^3).$$

Tak wysoki koszt obliczeniowy stanowi główną motywację do zastosowania programowania równoległego, pozwalającego rozdzielić pracę pomiędzy wiele procesów i skrócić czas wykonania programu.

3 Zrównoleglenie z wykorzystaniem UPC++

3.1 Model PGAS

UPC++ (Unified Parallel C++) jest biblioteką przeznaczoną do tworzenia aplikacji równoległych w modelu PGAS (Partitioned Global Address Space). Model ten łączy cechy pamięci współdzielonej i rozproszonej: każdy proces posiada własną pamięć lokalną, a wybrane obszary mogą być udostępniane globalnie.

Biblioteka UPC++ udostępnia między innymi:

- globalne wskaźniki (`global_ptr`),
- operacje jednostronnej komunikacji (`rput`, `rget`),
- operacje kolektywne (`broadcast`, `reduce_all`),
- bariery synchronizacyjne (`barrier`).

W przeciwieństwie do MPI komunikacja nie wymaga jawnych par operacji `send/recv`, lecz realizowana jest poprzez globalną przestrzeń adresową i operacje kolektywne.

3.2 Strategia zrównoleglenia

W projekcie zastosowano równoległe wykonywanie pojedynczego przebiegu algorytmu Dijkstry poprzez podział zbioru wierzchołków pomiędzy procesy.

Dla każdego źródła wszystkie procesy współpracują przy wyznaczaniu najkrótszych ścieżek. Rozwiązanie to dobrze odpowiada modelowi PGAS, ponieważ wymaga regularnej wymiany niewielkich ilości danych.

3.3 Podział danych

Dla p procesów każdy proces o numerze r otrzymuje blok wierzchołków:

$$V_r = \left[\frac{rn}{p}, \frac{(r+1)n}{p} \right).$$

Proces odpowiada za aktualizację elementów tablic `l` i `parent` należących do jego przedziału oraz wykonuje relaksację wyłącznie dla własnego fragmentu danych. Zapewnia to równomierny rozkład pracy i eliminuje konflikty zapisu.

3.4 Dystrybucja danych wejściowych

Macierz sąsiedztwa wczytywana jest przez proces 0, a następnie rozsyłana do wszystkich procesów przy użyciu operacji:

```
n = upcxx::broadcast(n, 0).wait();
```

```
upcxx::broadcast(  
    W.data(),  
    n*n,  
    0  
) .wait();
```

Każdy proces otrzymuje pełną kopię macierzy, dzięki czemu podczas relaksacji krawędzi nie jest wymagana dodatkowa komunikacja.

3.5 Globalna selekcja minimum

W każdej iteracji każdy proces wyszukuje minimum w swoim lokalnym bloku wierzchołków. Następnie wykonywana jest redukcja globalna:

```
DistVertex globalMin =
    upcxx::reduce_all(
        localMin,
        minloc
    ).wait();
```

Operacja ta wyznacza globalnie najmniejszą odległość oraz odpowiadający jej wierzchołek. Funkcjonalnie odpowiada to redukcji `MPI_MINLOC` stosowanej w MPI.

3.6 Relaksacja krawędzi

Po wyznaczeniu globalnego minimum każdy proces wykonuje relaksację krawędzi wyłącznie dla własnego przedziału wierzchołków.

Jeżeli dla krawędzi (u, v) spełniony jest warunek

$$l[u] + w(u, v) < l[v],$$

aktualizowane są wartości:

$$l[v] = l[u] + w(u, v), \quad \text{parent}[v] = u.$$

Po zakończeniu relaksacji rozpoczyna się kolejna iteracja algorytmu.

3.7 Agregacja wyników

Po zakończeniu przebiegu algorytmu proces 0 alokuje globalne tablice wynikowe:

```
gL = upcxx::new_array<int>(n);
gParent =
    upcxx::new_array<int>(n);
```

Każdy proces zapisuje swój fragment wyników przy pomocy operacji `rput()`:

```
upcxx::rput(
    l.data() + start,
    gL + start,
    blockLen
);
```

Dzięki wykorzystaniu pamięci globalnej nie jest konieczne stosowanie operacji typu `MPI_Gather`, ponieważ dane trafiają bezpośrednio do pamięci procesu docelowego.

3.8 Synchronizacja

Synchronizacja procesów realizowana jest za pomocą funkcji:

```
upcxx::barrier();
```

Bariery wykorzystywane są po rozesłaniu danych, podczas agregacji wyników oraz przed pomiarem czasu wykonania programu.

3.9 Złożoność i skalowanie

Dla pojedynczego przebiegu każdy proces wykonuje wyszukiwanie minimum oraz relaksację w czasie:

$$O\left(\frac{n}{p}\right).$$

Ponieważ wykonywanych jest około n iteracji, koszt obliczeń lokalnych wynosi:

$$O\left(\frac{n^2}{p}\right).$$

Dodatkowo w każdej iteracji wykonywana jest operacja `reduce_all()`, której koszt można oszacować jako:

$$O(\log p).$$

Całkowity czas wykonania dla pojedynczego źródła można więc zapisać jako:

$$T(n, p) = O\left(\frac{n^2}{p} + n \log p\right).$$

Dla problemu wszystkich źródeł otrzymujemy:

$$T(n, p) = O\left(\frac{n^3}{p} + n^2 \log p\right).$$

Zależność ta pokazuje, że wraz ze wzrostem liczby procesów maleje koszt obliczeń lokalnych, natomiast rośnie znaczenie kosztów komunikacyjnych.

4 Architektura projektu

4.1 Struktura projektu

Implementacja została przygotowana jako samodzielna aplikacja UPC++ składająca się z pliku źródłowego oraz pliku Makefile odpowiedzialnego za kompilację.

```
DijkstraUPCXX/  
|  
|-- src/  
|   '-- DijkstraUPCXX.cpp  
|-- Makefile  
|-- dane/  
|   |-- matrix_25.txt  
|   '-- ...  
'-- build/  
    '-- DijkstraUPCXX
```

Najważniejsze elementy projektu to implementacja algorytmu, przykładowe dane wejściowe oraz plik wykonywalny tworzony podczas kompilacji.

4.2 Struktura kodu

Kod programu można podzielić na kilka głównych części:

- obsługa argumentów programu,
- wczytywanie grafu,
- implementacja algorytmu Dijkstry,
- komunikacja UPC++,
- zapis wyników do pliku JSON,
- funkcja główna programu.

Najważniejsze funkcje przedstawiono w tabeli 1.

Tabela 1: Najważniejsze funkcje programu

Funkcja	Przeznaczenie
ImportGraphMatrix	Wczytanie macierzy sąsiedztwa.
ParseArgs	Obsługa argumentów programu.
ResolveOutputPath	Wyznaczenie ścieżki pliku JSON.
WriteJsonResult	Zapis wyników do pliku JSON.
PrintPath	Odtwarzanie ścieżki z tablicy poprzedników.
minloc	Operator redukcji dla <code>reduce_all()</code> .
DijkstraUPCXX	Równoległa implementacja algorytmu.
main	Główny przepływ programu.

4.3 Format danych wejściowych

Program przyjmuje graf zapisany w postaci macierzy sąsiedztwa. Wartości dodatnie oznaczają wagi krawędzi, natomiast wartość -1 oznacza brak połączenia.

Przykładowy fragment pliku:

```
-1  10  -1  16
10  -1   8  -1
-1   8  -1  12
16  -1  12  -1
```

Macierz przechowywana jest w jednowymiarowym wektorze, a dostęp do elementów realizowany jest przez indeks:

$$W[i \cdot n + j].$$

4.4 Format danych wyjściowych

Wyniki prezentowane są na standardowym wyjściu oraz zapisywane do pliku JSON.

Przykładowy fragment raportu tekstowego:

Shortest paths from source 0:

```
0 -> 0: 0 | path: 0
0 -> 1: 18 | path: 0 2 1
0 -> 2: 10 | path: 0 2
0 -> 3: 16 | path: 0 3
```

Jeżeli nie podano parametru `-output`, plik wynikowy tworzony jest obok pliku wejściowego.

4.5 Struktura pliku JSON

Plik wynikowy zawiera informacje o grafie, czasie wykonania programu oraz wynikach obliczeń.

```
{
  "n": 25,
  "num_processes": 8,
  "io_seconds": 0.004,
  "compute_seconds": 0.017,
  "elapsed_seconds": 0.021,
  "dist": [...],
  "parent": [...]
}
```

Najważniejsze pola opisują liczbę wierzchołków, liczbę procesów, czasy wykonania oraz macierze odległości i poprzedników. Wartość -1 oznacza brak ścieżki lub brak poprzednika.

4.6 Pomiar czasu wykonania

Program wykorzystuje bibliotekę `std::chrono` do pomiaru:

- czasu wejścia–wyjścia,
- czasu obliczeń,
- całkowitego czasu wykonania.

Rozdzielenie tych wielkości pozwala analizować zarówno koszt komunikacji i operacji wejścia–wyjścia, jak również wydajność samego algorytmu.

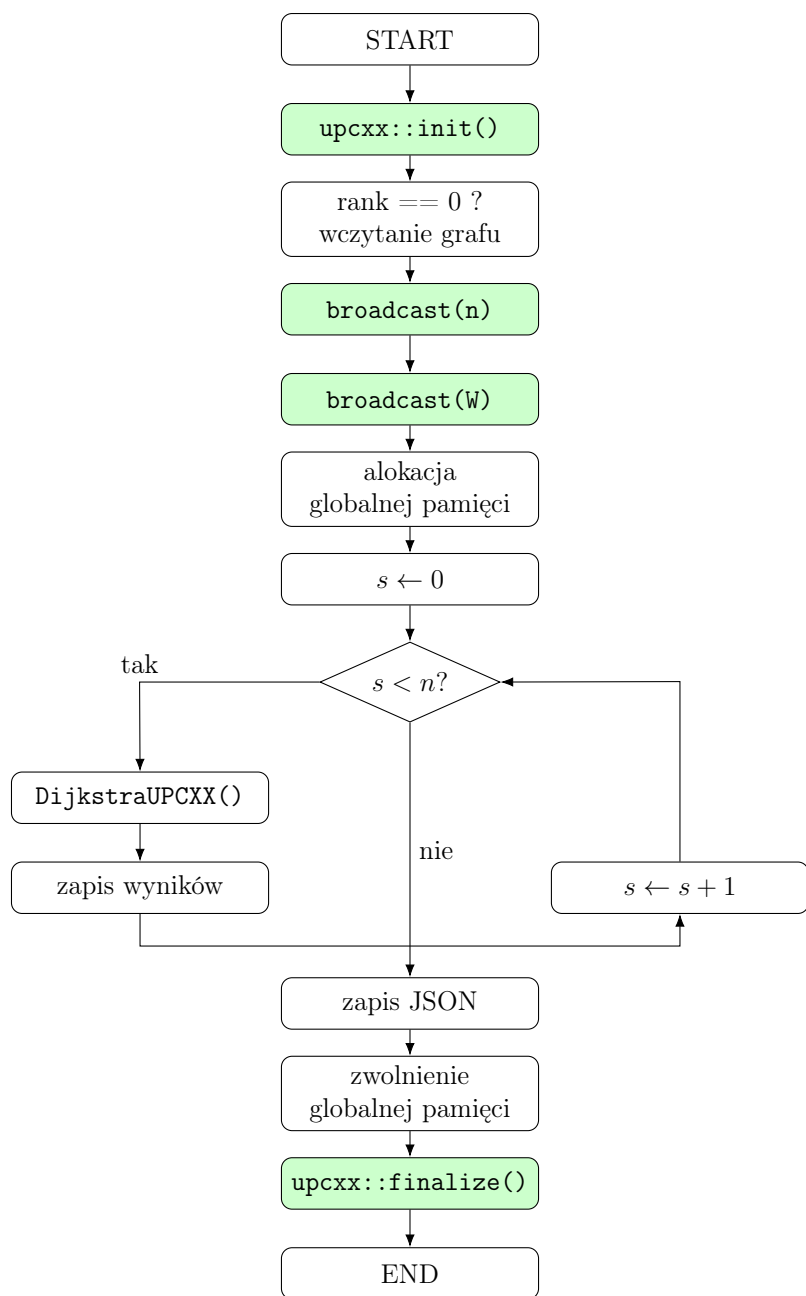
5 Schemat blokowy programu

W celu zobrazowania działania aplikacji przygotowano dwa schematy blokowe. Pierwszy przedstawia przebieg programu od inicjalizacji środowiska UPC++ do zapisania wyników i zakończenia działania. Drugi opisuje przebieg funkcji `DijkstraUPCXX()`, realizującej równoległy algorytm Dijkstry.

Kolorem zielonym oznaczono operacje komunikacyjne biblioteki UPC++.

5.1 Schemat głównej pętli programu

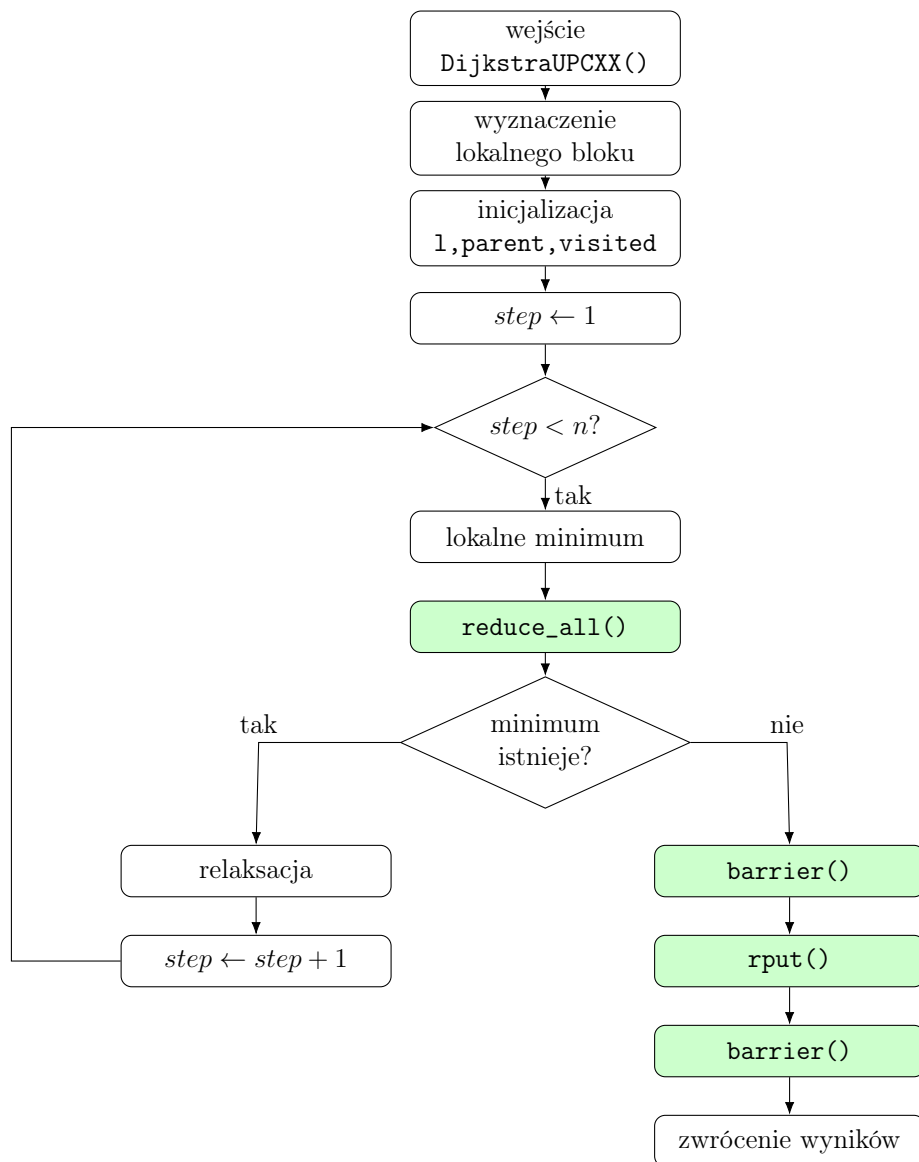
Rysunek 1 przedstawia przebieg funkcji `main()`. Program rozpoczyna działanie od inicjalizacji UPC++, następnie proces 0 wczytuje graf i rozsyła dane do pozostałych procesów. Kolejnym etapem jest wykonanie algorytmu Dijkstry dla wszystkich źródeł. Po zakończeniu obliczeń wyniki są zapisywane do pliku JSON, zwalniana jest pamięć globalna, a środowisko UPC++ zostaje zakończone.



Rysunek 1: Schemat blokowy głównej pętli programu

5.2 Schemat funkcji `DijkstraUPCXX()`

Rysunek 2 przedstawia przebieg pojedynczego wykonania równoległego algorytmu Dijkstry. Każdy proces wyznacza lokalny blok wierzchołków, wyszukuje lokalne minimum, uczestniczy w operacji `reduce_all()` oraz wykonuje relaksację dla własnego fragmentu danych. Po zakończeniu obliczeń wyniki są zapisywane do pamięci globalnej przy użyciu operacji `rput()`, a następnie zwracane do programu głównego.



Rysunek 2: Schemat blokowy funkcji DijkstraUPCXX()

6 Opis implementacji

6.1 Struktura kodu źródłowego

Cała implementacja programu znajduje się w pliku `DijkstraUPCXX.cpp`. Kod można podzielić na kilka głównych części:

- obsługę danych wejściowych i argumentów programu,
- funkcje pomocnicze,
- implementację równoległego algorytmu Dijkstry,
- zapis wyników do pliku JSON,
- funkcję `main()` sterującą działaniem programu.

Program wykorzystuje bibliotekę UPC++, kontenery STL oraz mechanizmy obsługi plików i pomiaru czasu dostępne w standardzie C++17.

6.2 Struktury pomocnicze

Do wyznaczania globalnego minimum wykorzystywana jest struktura:

```
struct DistVertex
{
    int dist;
    int vertex;
};
```

Przechowuje ona aktualną odległość oraz numer odpowiadającego jej wierzchołka.

Na potrzeby operacji `reduce_all()` zdefiniowano funkcję `minloc()`, która wybiera element o najmniejszej odległości, a w przypadku remisu wierzchołek o mniejszym numerze. Pełni ona rolę odpowiednika operacji `MINLOC` znanej z MPI.

6.3 Wczytywanie danych i zapis wyników

Macierz sąsiedztwa wczytywana jest przez funkcję `ImportGraphMatrix()`, która odczytuje plik tekstowy i zapisuje graf w jednowymiarowym wektorze:

$$W[i \cdot n + j].$$

Takie rozwiązanie upraszcza rozsyłanie danych pomiędzy procesami oraz zapewnia ciągły układ danych w pamięci.

Program obsługuje również opcjonalny parametr `-output`, umożliwiający wskazanie lokalizacji pliku wynikowego.

Wyniki zapisywane są przez funkcję `WriteJsonResult()`, która generuje plik JSON zawierający macierze odległości i poprzedników oraz informacje o czasie wykonania programu.

6.4 Implementacja algorytmu Dijkstry

Najważniejszym elementem programu jest funkcja `DijkstraUPCXX()`, realizująca pojedynczy przebieg algorytmu dla wybranego źródła.

Na początku każdy proces wyznacza własny zakres wierzchołków:

```
int start = rank * n / size;  
int end   = (rank + 1) * n / size;
```

Następnie tworzone są lokalne tablice odległości, poprzedników oraz znaczników odwiedzenia.

W każdej iteracji algorytmu wykonywane są trzy podstawowe kroki:

1. wyszukanie lokalnego minimum,
2. wyznaczenie minimum globalnego przy pomocy `reduce_all()`,
3. relaksacja krawędzi dla lokalnego bloku wierzchołków.

Po zakończeniu relaksacji rozpoczyna się kolejna iteracja, aż do przetworzenia wszystkich osiągalnych wierzchołków.

6.5 Agregacja wyników

Po zakończeniu przebiegu algorytmu procesy zapisują swoje fragmenty wyników do pamięci globalnej przy użyciu operacji:

```
upcxx::rput (...)
```

Proces 0 odczytuje następnie kompletne tablice odległości i poprzedników, które zostają zapisane do struktur wynikowych.

Zastosowanie pamięci globalnej pozwala uniknąć dodatkowych operacji zbierania danych charakterystycznych dla MPI.

6.6 Funkcja główna programu

Funkcja `main()` odpowiada za inicjalizację środowiska UPC++, wczytanie danych wejściowych, rozesłanie macierzy grafu oraz uruchomienie algorytmu dla wszystkich źródeł.

Po zakończeniu obliczeń wyniki są zapisywane do pliku JSON, zwalniana jest pamięć globalna, a środowisko UPC++ zostaje zakończone funkcją:

```
upcxx::finalize();
```


7 Budowa i obsługa programu

7.1 Wymagania i kompilacja

Program został zaimplementowany w języku C++ z wykorzystaniem biblioteki UPC++. Do jego kompilacji wymagane są:

- kompilator UPC++,
- środowisko GASNet,
- narzędzie `make`.

Proces budowania projektu został zautomatyzowany przy pomocy pliku `Makefile`. Kompilacja wykonywana jest poleceniem:

```
make
```

lub równoważnie:

```
make all
```

Polecenie tworzy katalog `build/` oraz kompiluje program do pliku:

```
build/DijkstraUPCXX
```

W procesie kompilacji wykorzystywany jest kompilator UPC++ z optymalizacją `-O2` oraz standardem języka C++17.

7.2 Uruchamianie programu

Plik `Makefile` udostępnia dwa sposoby uruchamiania programu.

Uruchomienie w środowisku laboratoryjnym realizowane jest poleceniem:

```
make run
```

Polecenie automatycznie generuje listę węzłów, a następnie uruchamia program przy użyciu narzędzia `upcxx-run` z liczbą procesów określoną przez zmienną:

```
NP = 4
```

Dla testów wykonywanych na pojedynczym komputerze dostępna jest również reguła:

```
make run-local
```

W tym przypadku wszystkie procesy uruchamiane są lokalnie z wykorzystaniem opcji `-localhost`.

Domyślnie program wykonywany jest dla pliku:

```
data/matrix_25.txt
```

oraz wykorzystuje współdzielony obszar pamięci o rozmiarze:

```
HEAP = 256M
```

7.3 Dodatkowe reguły Makefile

Plik Makefile zawiera również reguły pomocnicze:

- `make clean` – usunięcie katalogu `build/` oraz pliku z listą węzłów,
- `make reset` – pełne wyczyszczenie projektu (alias reguły `clean`).

Pozwala to szybko przygotować środowisko do ponownej kompilacji i uruchomienia programu.

7.4 Wyniki działania

Podczas działania proces 0 prezentuje długości najkrótszych ścieżek oraz odpowiadające im trasy.

Przykładowy fragment wyjścia:

```
Shortest paths from source 0:
```

```
0 -> 0: 0 | path: 0
0 -> 1: 10 | path: 0 1
0 -> 2: 20 | path: 0 1 2
```

Po zakończeniu obliczeń wyświetlane jest podsumowanie zawierające podstawowe informacje o wykonaniu programu:

```
[JSON] wrote matrix_25.json
(n=25, np=8,
 io=0.004s,
 compute=0.017s,
 total=0.021s)
```

Wyniki zapisywane są również do pliku JSON.

7.5 Obsługa błędów

Program wykorzystuje mechanizm wyjątków języka C++. Najczęstsze sytuacje błędne to brak pliku wejściowego, niepoprawne argumenty programu lub brak możliwości zapisania pliku wynikowego.

Przykładowe komunikaty:

```
Error: matrix path is required
```

```
Failed to load matrix:
Could not open file
```

7.6 Przebieg działania programu

Działanie programu można podzielić na następujące etapy:

1. inicjalizacja środowiska UPC++,
2. wczytanie i rozesłanie danych wejściowych,
3. wykonanie algorytmu Dijkstry dla wszystkich źródeł,
4. agregacja wyników,
5. zapis wyników do pliku JSON,
6. zwolnienie zasobów i zakończenie programu.

8 Przykłady działania i analiza wyników

8.1 Przykładowe dane i uruchomienie

Do weryfikacji poprawności programu wykorzystano graf zapisany w postaci macierzy sąsiedztwa:

```
-1 10 -1 16
10 -1 8 -1
-1 8 -1 12
16 -1 12 -1
```

Program uruchomiono przy użyciu ośmiu procesów UPC++:

```
upcxx-run -n 8 \
./DijkstraUPCXX matrix_25.txt
```

Po uruchomieniu proces 0 wczytuje dane wejściowe, rozsyła je do pozostałych procesów, wykonuje obliczenia oraz zapisuje wyniki do pliku JSON.

8.2 Przykładowe wyniki

Dla każdego źródła program wyświetla długości najkrótszych ścieżek oraz odpowiadające im trasy.

Przykładowy fragment wyjścia:

Shortest paths from source 0:

```
0 -> 0: 0 | path: 0
0 -> 1: 10 | path: 0 1
0 -> 2: 20 | path: 0 1 2
0 -> 3: 16 | path: 0 3
```

Po zakończeniu obliczeń tworzony jest plik JSON zawierający czasy wykonania programu oraz macierze odległości i poprzedników.

```
{
  "n": 25,
  "num_processes": 8,
  "io_seconds": 0.004,
  "compute_seconds": 0.017,
  "elapsed_seconds": 0.021,
  "dist": [...],
  "parent": [...]
}
```

8.3 Analiza komunikacji i skalowalności

Najważniejszym elementem komunikacyjnym programu jest operacja `reduce_all()`, wykonywana w każdej iteracji algorytmu w celu wyznaczenia globalnego minimum. Pozostałe operacje komunikacyjne (`broadcast()`, `rput()` oraz `barrier()`) wykonywane są znacznie rzadziej.

Przybliżony czas wykonania pojedynczego przebiegu algorytmu można opisać zależnością:

$$T(n, p) = O\left(\frac{n^2}{p} + n \log p\right).$$

Wraz ze wzrostem liczby procesów maleje koszt obliczeń lokalnych, natomiast rośnie znaczenie kosztów komunikacyjnych. Dla większych grafów możliwe jest uzyskanie zauważalnego przyspieszenia wykonania.

8.4 Porównanie z implementacją MPI

Implementacja UPC++ realizuje tę samą logikę obliczeń co wersja MPI, jednak wykorzystuje inny model komunikacji.

W MPI komunikacja opiera się głównie na operacjach kolektywnych, takich jak `MPI_Bcast`, `MPI_Allreduce` czy `MPI_Gather`. W UPC++ część tej funkcjonalności realizowana jest poprzez globalną przestrzeń adresową oraz operacje jednostronne.

Zastosowanie `rput()` umożliwia bezpośredni zapis wyników do pamięci procesu docelowego, co upraszcza implementację i dobrze odzwierciedla model PGAS wykorzystywany przez UPC++.

9 Podsumowanie

Celem projektu było opracowanie równoległej implementacji algorytmu Dijkstry z wykorzystaniem biblioteki UPC++ oraz modelu programowania PGAS.

Zrealizowany program umożliwia wyznaczanie drzew najkrótszych ścieżek dla wszystkich wierzchołków źródłowych grafu ważonego zapisanego w postaci macierzy sąsiedztwa. Implementacja obejmuje wczytywanie danych, równoległe wykonywanie algorytmu, agregację wyników oraz ich zapis do formatu JSON.

Do komunikacji wykorzystano podstawowe mechanizmy biblioteki UPC++, takie jak `broadcast()`, `reduce_all()`, `rput()` oraz `barrier()`. Kluczowym elementem algorytmu jest operacja globalnej redukcji minimum, zapewniająca poprawność równoległego wykonania.

Przeprowadzona analiza wykazała, że wraz ze wzrostem liczby procesów maleje koszt obliczeń lokalnych, natomiast coraz większe znaczenie zyskują koszty komunikacyjne. Zastosowanie modelu PGAS pozwoliło uprościć agregację wyników dzięki wykorzystaniu globalnej przestrzeni adresowej i operacji jednostronnych.

Implementacja UPC++ realizuje tę samą logikę obliczeń co wersja MPI, jednak wykorzystuje odmienny model komunikacji. Pozwoliło to na praktyczne porównanie podejścia komunikatowego z modelem PGAS oraz zapoznanie się z możliwościami biblioteki UPC++ w zastosowaniach związanych z programowaniem równoległym.